

CERTIFICATION CONCEPTEUR DÉVELOPPEUR D'APPLICATIONS

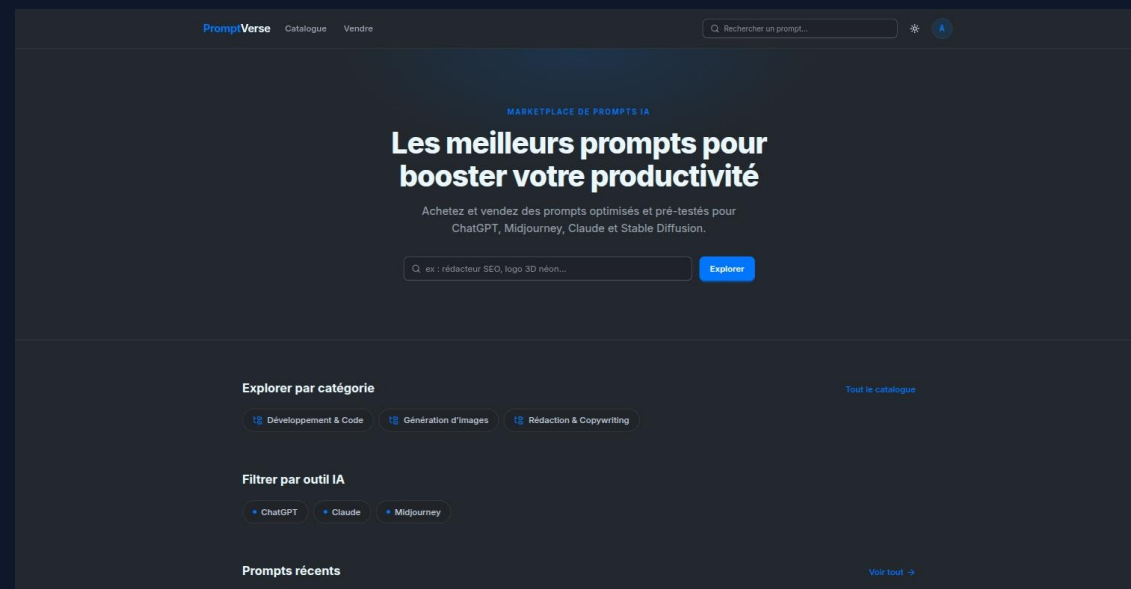
PromptVerse

La marketplace de prompts pour les IA génératives

promptverse.biz

LAMALLEM AMINE

Simplon · Formation CDA · 2026



Introduction



Amine LAMALLEM

Apprenant Concepteur Développeur d'Applications

Simplon · 2025 – 2026



Titre visé

Concepteur Développeur d'Applications — niveau 6



Projet chef-d'œuvre

PromptVerse, réalisé en équipe de trois



En production

<https://promptverse.biz>

L'équipe



Abduljaber

Conception · Développement



Amine

Conception · Développement



Anis

Conception · Développement



Formateur Simplon : briefs de sprint, suivi et validation des livrables

PromptVerse, c'est quoi ?

Une marketplace où chacun peut vendre ses prompts optimisés pour les IA génératives, et acheter ceux des autres pour gagner du temps.



Catalogue

Prompts filtrables par catégorie et par outil IA (ChatGPT, Midjourney, Claude...)



Publication

Tout utilisateur publie ses prompts avec prix, aperçu et images



Achat

Paieement Stripe Checkout ; les prompts à 0 € sont gratuits



Administration

Gestion des utilisateurs, des rôles, des catégories et des outils IA

Règle clé : le contenu payant d'un prompt reste invisible tant que l'achat n'est pas confirmé.

À qui s'adresse la plateforme ?

Une communauté digitale, créative et orientée productivité (20–40 ans).



Créateurs & experts

Créateurs de contenu, freelances et professionnels du web qui veulent rentabiliser leur expertise en ingénierie de prompts.



Entrepreneurs & pros

Petites entreprises et indépendants qui cherchent des prompts prêts à l'emploi pour gagner du temps au quotidien.



Early adopters

Passionnés d'IA qui testent les nouveaux outils et veulent des exemples fiables pour progresser.

Sommaire

01 Organisation et outils

02 Conception

03 Architecture et back-end

04 Front-end

05 Sécurité

06 Tests

07 DevOps et déploiement

08 Déroulement et démo

Stack et outils

Front-end



React 19



TypeScript



Vite



TanStack Query



Tailwind + daisyUI



React Router

Back-end



NestJS 11



TypeORM



MySQL 8.4



JWT + bcrypt



Stripe

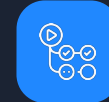


Nodemailer / SMTP

DevOps et outils



Docker



GitHub Actions



Nginx



Figma



Swagger



Git

Communication et organisation



Scrum

Méthodologie

8 sprints avec revue à chaque fin



Trello

User stories

Backlog par sprint, critères d'acceptation



GitHub

Versioning

Branches dev / main / deploy



OpenSpec

Spécifications

Une proposition versionnée par module

02

Conception

Des besoins aux modèles, avant la première ligne de code



User stories et rôles

USER

Rôle utilisateur

- Parcourir et acheter des prompts
- Publier ses propres contenus
- Gérer son profil, ses ventes et ses achats

ADMIN

Rôle administrateur

- Toutes les fonctionnalités USER
- Gérer les catégories et les outils IA
- Consulter la liste des utilisateurs

SUPER_ADMIN

Rôle super administrateur

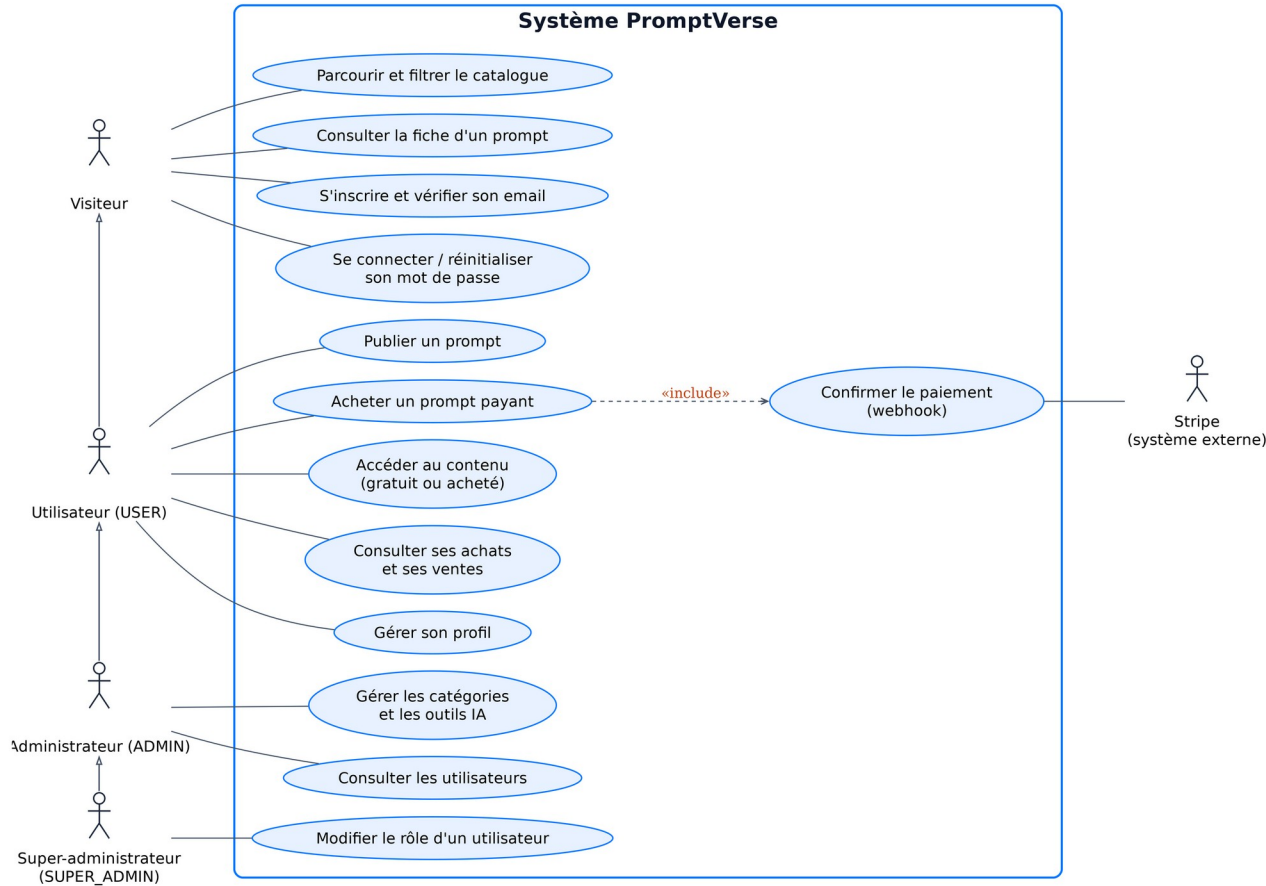
- Toutes les permissions ADMIN
- Modifier le rôle d'un utilisateur
- Seul rôle habilité à promouvoir un admin

[openspec/changes/module-orders-stripe-fullstack/specs/payment-processing/spec.md](#) — critère d'acceptation (Gherkin)

Scenario: Checkout Session Creation

- ****GIVEN**** an authenticated user with items in their cart
- ****WHEN**** calling `POST /orders/checkout`
- ****THEN**** the system **MUST** create an `Order` with status `PENDING`, create `OrderItem` records, clear the user's cart, and return a Stripe Checkout URL.

Diagramme de cas d'utilisation

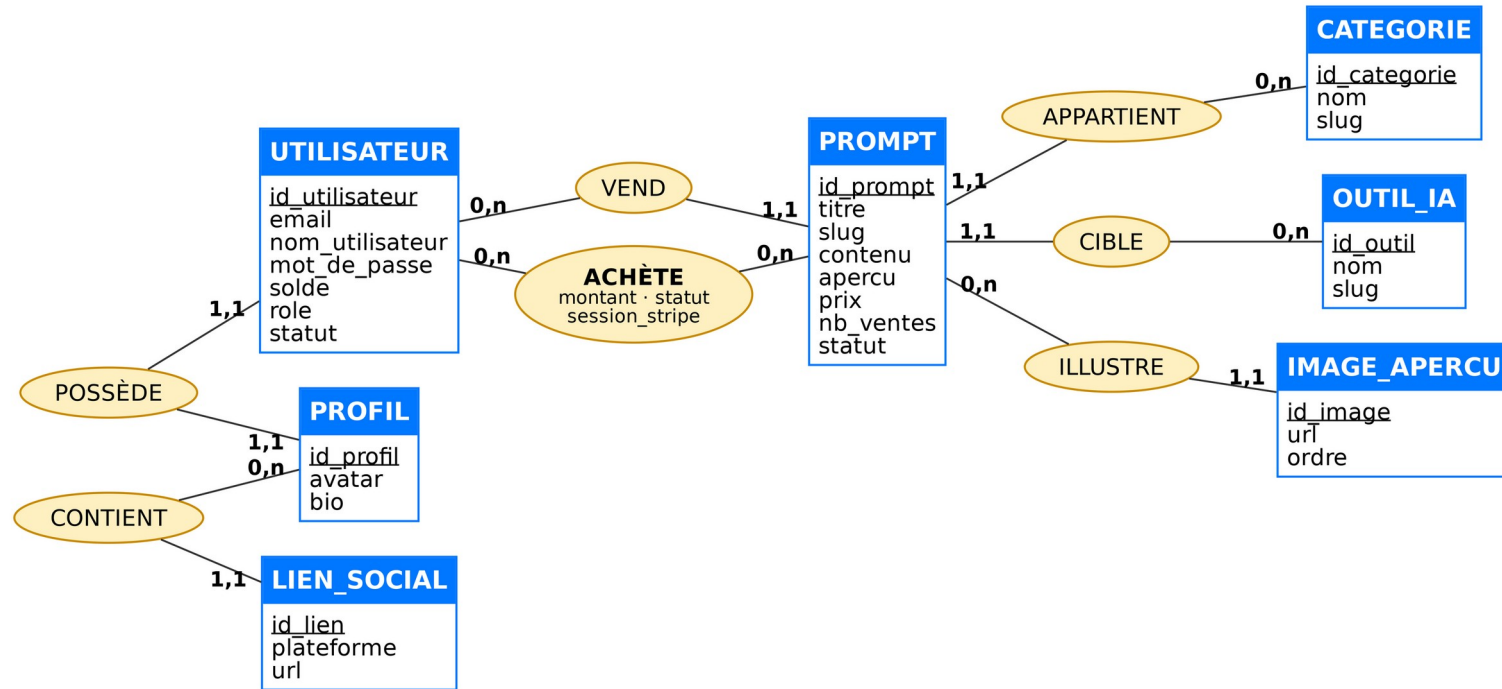


4 acteurs + Stripe (système externe)

Héritage visiteur → USER → ADMIN
→ SUPER_ADMIN

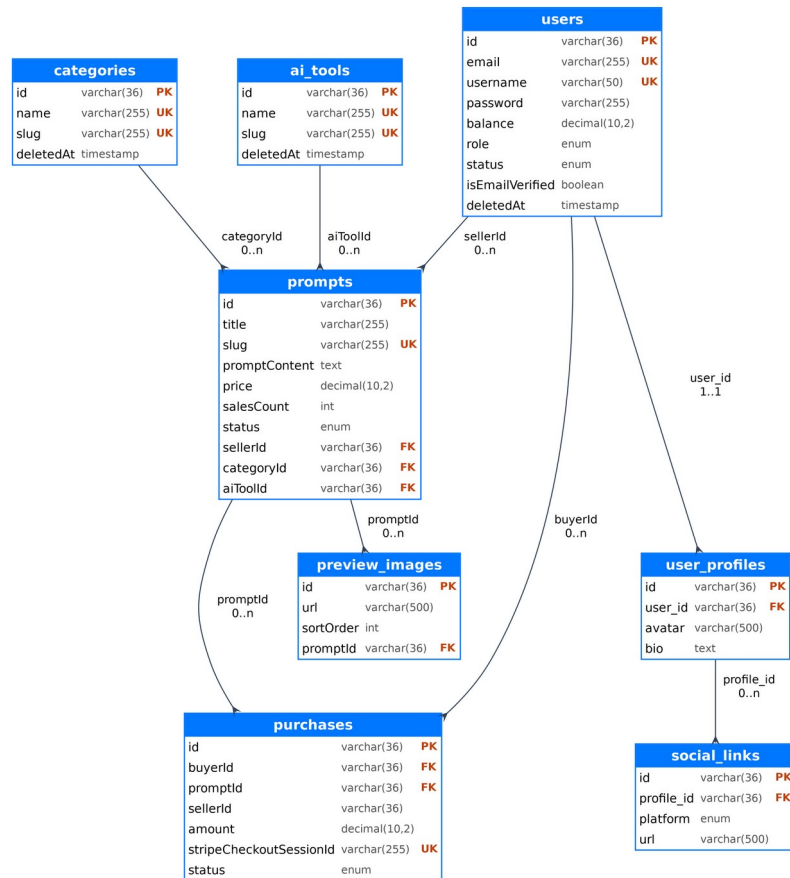
«include» l'achat inclut la confirmation
par webhook

Modèle conceptuel de données



ACHÈTE porte ses propres données (montant, statut, session Stripe) → devient la table purchases dans le MLD.

Modèle physique de données



8 tables MySQL

structure relationnelle normalisée

Identifiants UUID

clés primaires non devinables (v4)

Montants DECIMAL(10,2)

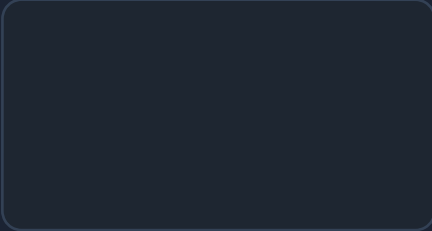
précision exacte pour les paiements

Migrations TypeORM

le schéma est versionné avec le code

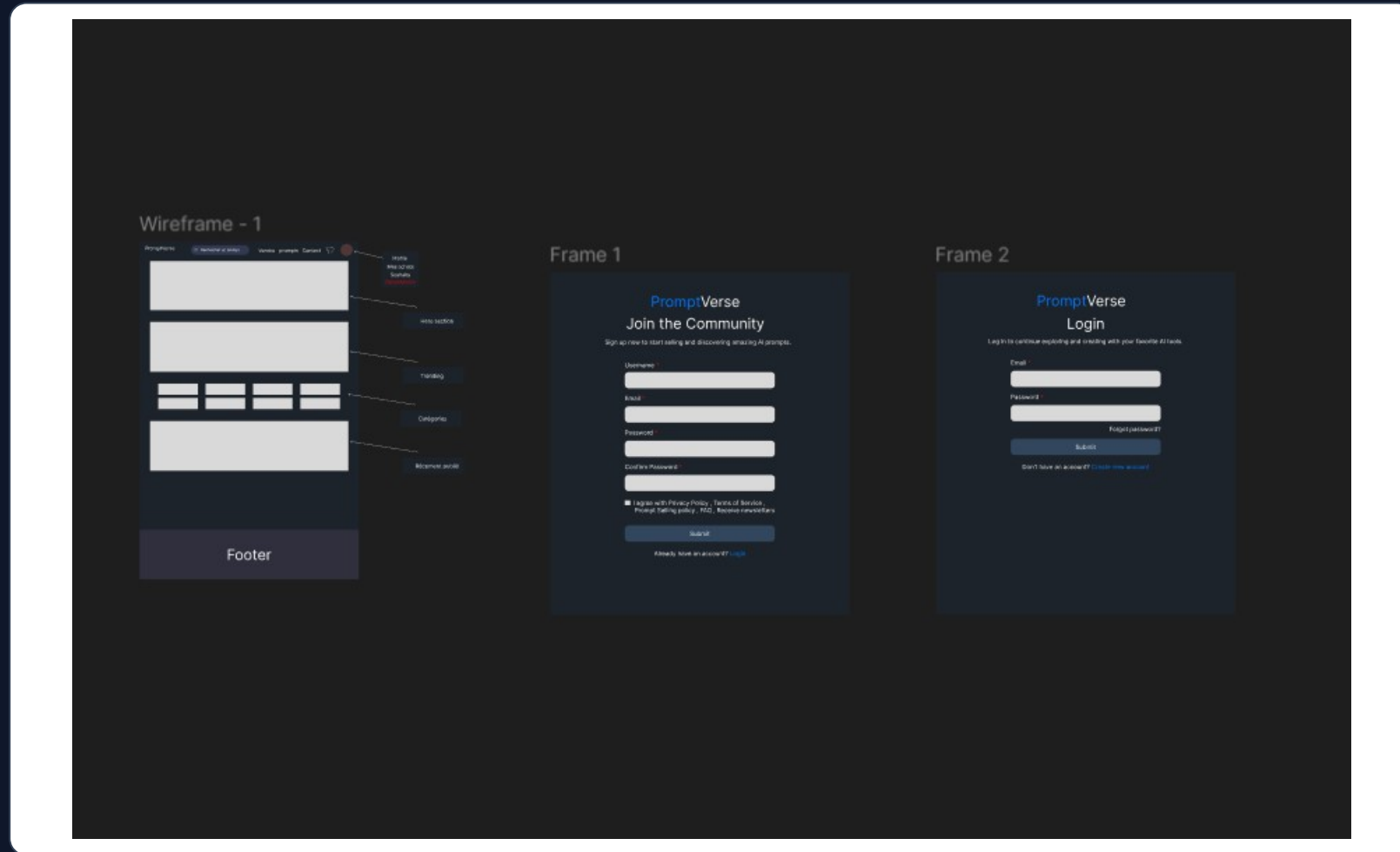
Charte graphique et accessibilité

Palette du thème sombre, vérifiée avec les ratios de contraste WCAG.

			
#0E1217	#1E2631	#6366F1	#0077FF
Fond sombre	Surface / cartes	Accent violet	Action / primaire
Ratio 17.5:1 (AAA)	Ratio 11.2:1 (AAA)	Ratio 4.6:1 (AA)	Ratio 4.8:1 (AA)

Typographie Inter · composants daisyUI · interface responsive du mobile au grand écran.

Maquette de l'interface



Maquettes réalisées sous Figma

- Wireframe basse fidélité pour placer les zones de la page
- Maquettes des écrans d'inscription et de connexion
- Thème sombre, accent bleu, composants réutilisables
- Les captures de la section Front-end montrent le résultat réel

Open Spec & Context IA



1. Définition

- OpenSpec est un outil de spec-driven development : la fonctionnalité est décrite dans des fichiers versionnés avant d'être codée.
- Un dossier par module : proposal.md (le besoin), design.md, specs/ (les critères) et tasks.md (le découpage).
- Le Context IA est le bloc context: de config.yaml : il donne au modèle le projet, la stack et les règles métier.



2. Pourquoi & Comment

POURQUOI

- Spécifier avant de coder évite d'implémenter une interprétation du besoin.
- Les critères d'acceptation deviennent la liste des tests.
- Le contexte partagé garde l'IA dans notre stack et nos règles.

COMMENT

- Une proposition versionnée par module, à côté du code.
- Critères écrits en Gherkin : GIVEN / WHEN / THEN.
- Règles déclarées une seule fois dans config.yaml.



3. Exemple concret

- config.yaml : schéma, contexte projet et règles de rédaction.
- spec.md : les scénarios de paiement et de déblocage du contenu.
- tasks.md : les tâches numérotées, cochées au fur et à mesure.
- Voir les deux diapositives suivantes.

Open Spec : le contexte donné à l'IA

openspec/config.yaml (extrait)

```
schema: spec-driven

# Project Context for OpenSpec & AI Assistants
context: |
  Project: PromptVerse – Marketplace de Prompts IA (ChatGPT, Midjourney, DALL-E, Claude,
  etc.)
  Certification: Titre Professionnel CDA (Concepteur Développeur d'Applications - RNCP 37873)

  Architecture & Domain Rules:
    - 3 Roles: USER (Achète ET Vend des prompts), ADMIN (Gestion & Modération),
      SUPER_ADMIN (Administration Système).
    - Publication Directe: Les prompts créés par un USER passent immédiatement au statut
      'PUBLISHED' (ou 'ARCHIVED'). Pas de statut DRAFT ni PENDING.
    - Confidentialité du Contenu: Le champ 'promptContent' est masqué et accessible
      UNIQUEMENT après validation d'une commande.

rules:
  proposal:
    - "Focus on Spec-Driven Development: clear goals, scope, and non-goals."
    - "Always keep security (OWASP) and JWT RBAC permissions in mind."
  specs:
    - "Write functional requirements with Given-When-Then (Gherkin) acceptance criteria."
  tasks:
    - "Break implementation into small, testable chunks (module by module)."
```



Ce que ça change

- L'IA connaît les rôles, les règles métier et la stack du projet
- Les réponses restent dans notre architecture
- Chaque proposition de l'IA est relue, testée et validée par nous
- La décision finale reste humaine

Remarque : ce fichier date du cadrage ; certains choix ont évolué depuis (Redis a été retiré, le stockage est passé sur disque local).

Open Spec : de la spécification aux tâches

openspec/changes/module-orders-stripe-fullstack/proposal.md (extrait)

Why

Implement the shopping cart and checkout process to allow users to purchase prompts, generating revenue for sellers. This integrates Stripe Checkout for processing payments.

What Changes

- **Backend**: Integrate Stripe SDK to create Checkout Sessions (`POST /orders/checkout`).
- **Backend**: Implement Stripe Webhook (`POST /stripe/webhook`) to handle `checkout.session.completed` events and mark orders as `PAID`.

Modified Capabilities

- `content-masking`: Modification of the Prompt retrieval logic to return the full

tasks.md (extrait)

2. Backend Stripe Integration

- [] 2.1 Install `stripe` SDK.
- [] 2.2 Implement `POST /orders/checkout` to build line items and return a Stripe Session URL.
- [] 2.3 Implement `POST /stripe/webhook` with raw body parsing to validate signatures.
- [] 2.4 Add logic to update `Order` status to `PAID` on webhook success.



De la spec au code

La spécification parlait de panier et de commandes. À l'implémentation, nous avons simplifié : un achat = un prompt, dans la table purchases. La spec a servi de base de discussion, pas de contrat figé.

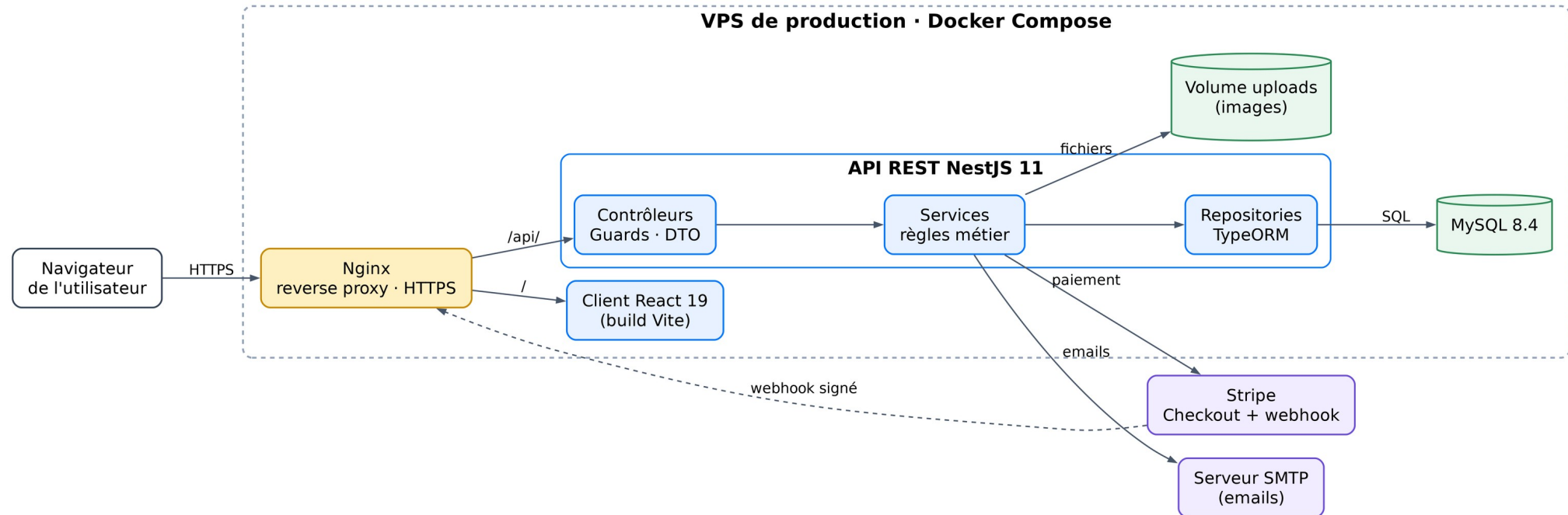
03

Architecture et back-end

API REST NestJS organisée en couches



Architecture de l'application



Découplée : le front ne parle qu'à l'API

Un seul point d'entrée : Nginx en HTTPS

Sécurité centralisée : toutes les règles côté serveur

Circulation d'une requête dans l'API



Exemple : l'achat d'un prompt

```
POST /api/v1/purchases/checkout-session { "promptId": "..." }  
→ AuthGuard (cookie JWT) → ValidationPipe (CreateCheckoutSessionDto : UUID)  
→ PurchasesController.createCheckoutSession()  
→ PurchasesService (règles métier) → Repository<Purchase> → MySQL
```

Authentification : JWT en cookie httpOnly

apps/api/src/modules/auth/auth.controller.ts

```
@Post('login')
@Throttle({
  short: { limit: 2, ttl: 1000 },
  medium: { limit: 5, ttl: 60000, blockDuration: 15 * 60 * 1000 },
  long: { limit: 20, ttl: 3600000 },
})
@Public()
@HttpCode(HttpStatus.OK)
async login(
  @Body() loginDto: LoginDto,
  @Res({ passthrough: true }) res: Response,
) {
  const { accessToken } = await this.authService.login(loginDto);

  res.cookie('access_token', accessToken, {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'lax',
    maxAge: 1000 * 60 * 60 * 24,
  });

  return { message: 'Login successful' };
}
```

bcrypt coût 10, mot de passe jamais renvoyé

Email vérifié obligatoire pour se connecter

Cookie httpOnly protège le jeton en cas de XSS

Guard global toute route est privée sauf @Public()

Throttling 5 essais/min puis blocage 15 min

Entité Prompt (TypeORM)

apps/api/src/modules/prompts/entities/prompt.entity.ts (extrait)

```
@Entity('prompts')
export class Prompt {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ type: 'varchar', length: 255 })
  title: string;

  @Column({ type: 'varchar', length: 255, unique: true })
  slug: string;

  @Column({ type: 'text' })
  promptContent: string;

  @Column({ type: 'decimal', precision: 10, scale: 2 })
  price: string;

  @Column({ type: 'enum', enum: PromptStatus, default: PromptStatus.PUBLISHED })
  status: PromptStatus;

  @ManyToOne(() => User, (user) => user.prompts, { onDelete: 'RESTRICT' })
  @JoinColumn({ name: 'sellerId' })
  seller: User;
}
```

Une classe = une table. L'entité décrit les colonnes et les relations.

slug unique : l'URL publique du prompt.

decimal(10,2) : pas d'erreur d'arrondi sur les prix.

RESTRICT : impossible de supprimer un vendeur qui a des prompts, l'historique est préservé.

Du code TypeORM à la requête SQL

apps/api/src/modules/prompts/prompts.service.ts

```
findAll(pageNumber: number, promptsPerPage: number) {  
  return this.promptsRepository.find({  
    select: PromptsService.LIST_SELECT,  
    where: { status: PromptStatus.PUBLISHED },  
    skip: promptsPerPage * (pageNumber - 1),  
    take: promptsPerPage,  
    order: { createdAt: 'DESC' },  
  });  
}
```

Requête SQL générée par TypeORM (page 2 du catalogue)

```
SELECT id, title, slug, previewResult, coverImage,  
       price, salesCount, status, createdAt,  
       sellerId, categoryId, aiToolId  
FROM prompts  
WHERE status = 'PUBLISHED'  
ORDER BY createdAt DESC  
LIMIT 10 OFFSET 10;
```



Pourquoi passer par l'ORM ?

- Les valeurs sont envoyées en paramètres : pas d'injection SQL possible
- La colonne promptContent n'est même pas demandée dans les listes
- Le typage TypeScript protège des fautes de nom de colonne
- Le schéma évolue par migrations versionnées, pas à la main

Règle métier : protéger le contenu payant

prompts.service.ts — findOneBySlug()

```
const isOwner = currentUserId === prompt.sellerId;
const isFree = PromptsService.isFree(prompt.price);
const hasPurchased =
  !isOwner && !isFree && currentUserId
    ? await this.hasCompletedPurchase(currentUserId, prompt.id)
    : false;
const canViewFullContent = isOwner || isFree || hasPurchased;

return {
  ...prompt,
  promptContent: canViewFullContent ? prompt.promptContent : null,
  isPurchasedByCurrentUser: hasPurchased,
};
```

Qui voit le contenu ?

Visiteur **null**

Utilisateur sans achat **null**

Vendeur **visible**

Achat COMPLETED **visible**

Prompt gratuit (0 €) **visible**

Et dans les listes du catalogue, la colonne n'est même pas lue en base (sélection explicite des colonnes).

Pairement Stripe et webhook idempotent



1. Définition

- Stripe Checkout : une page de paiement hébergée par Stripe ; la carte n'est jamais saisie sur notre site.
- Webhook : requête HTTP envoyée par Stripe à notre API pour annoncer `checkout.session.completed`.
- Idempotent : rejouer le même événement donne le même résultat qu'une seule exécution.



2. Pourquoi & Comment

POURQUOI

- Aucune donnée bancaire chez nous : la conformité PCI-DSS reste chez Stripe.
- Le navigateur n'est pas fiable : seul le webhook signé valide un achat.
- Stripe peut rejouer un événement : sinon le vendeur serait payé deux fois.

COMMENT

- Règles métier vérifiées avant tout appel à Stripe.
- Achat PENDING, puis session Checkout signée.
- Si l'achat est déjà COMPLETED, on ne fait rien.



3. Exemple concret

- `createCheckoutSession()` : les règles métier et la création de la session.
- `handleWebhookEvent()` / `completePurchase()` : la confirmation idempotente.
- Test unitaire dédié à l'idempotence.
- Voir les deux diapositives suivantes.

Stripe : création de la session de paiement

purchases.service.ts — createCheckoutSession() (1/2)

```
async createCheckoutSession(
  buyerId: string,
  promptId: string,
): Promise<{ url: string }> {
  const prompt = await this.promptsService.findByIdForPurchase(promptId);

  // Un prompt gratuit se débloque directement à la lecture
  // (PromptsService.findOneBySlug) – jamais via Stripe.
  if (PromptsService.isFree(prompt.price)) {
    throw new BadRequestException({
      code: ErrorCodes.PROMPT_IS_FREE,
    });
  }

  if (prompt.sellerId === buyerId) {
    throw new BadRequestException({
      code: ErrorCodes.CANNOT_PURCHASE_OWN_PROMPT,
    });
  }

  const alreadyPurchased = await
  this.promptsService.hasCompletedPurchase(
    buyerId,
    promptId,
  );
  if (alreadyPurchased) {
    throw new ConflictException({
      code: ErrorCodes.PROMPT_ALREADY_PURCHASED,
    });
  }
}
```

purchases.service.ts (2/2)

```
// Réutilise une tentative PENDING existante plutôt que
// d'empiler les
// lignes à chaque nouveau clic sur "Acheter" (retry après
// abandon).
let purchase = await this.purchasesRepository.findOne({
  where: { buyerId, promptId, status:
PurchaseStatus.PENDING },
});

purchase ??= await this.purchasesRepository.save(
  this.purchasesRepository.create({
    buyerId,
    promptId,
    sellerId: prompt.sellerId,
    amount: prompt.price,
    stripeCheckoutSessionId: `pending-${buyerId}-${promptId}-${
Date.now()}`,
    status: PurchaseStatus.PENDING,
  }),
);

session = await this.stripeService.createCheckoutSession({
  mode: 'payment',
  line_items: [{ quantity: 1, price_data: {
    currency: 'eur',
    unit_amount: PurchasesService.eurosToCents(prompt.price),
    product_data: { name: prompt.title } } }],
  success_url: `${clientUrl}/purchases/success?
session_id={CHECKOUT_SESSION_ID}`,
  cancel_url: `${clientUrl}/prompts/${prompt.slug}`,
  client_reference_id: buyerId,
  metadata: { buyerId, promptId, purchaseId: purchase.id },
});
```

Stripe : webhook signé et idempotent

purchases.controller.ts

```
@Post('webhook')
@Public()
@SkipThrottle()
@HttpCode(HttpStatus.OK)
async handleWebhook(
  @Req() req: RawBodyRequest<Request>,
  @Headers('stripe-signature') signature: string | undefined,
) {
  if (!req.rawBody || !signature) {
    throw new BadRequestException('Missing Stripe signature or body');
  }

  await this.purchasesService.handleWebhookEvent(req.rawBody, signature);

  return { received: true };
}
```



Signature vérifiée sur le corps brut

NestJS parse le JSON de toutes les requêtes. L'option `rawBody` garde le texte exact envoyé par Stripe, seul moyen de recalculer la signature.

purchases.service.ts — completePurchase()

```
private async completePurchase(session: Stripe.Checkout.Session) {
  const purchase = await this.purchasesRepository.findOne({
    where: { stripeCheckoutSessionId: session.id },
  });

  if (!purchase) {
    this.logger.warn(
      `Aucun achat trouvé pour la session Stripe ${session.id}`,
    );
    return;
  }

  // Idempotence : Stripe peut renvoyer le même événement plusieurs fois.
  if (purchase.status === PurchaseStatus.COMPLETED) {
    return;
  }

  purchase.status = PurchaseStatus.COMPLETED;
  await this.purchasesRepository.save(purchase);

  await this.promptsService.incrementSalesCount(purchase.promptId);

  const seller = await this.usersService.findById(purchase.sellerId);
  if (seller) {
    seller.balance = Number(seller.balance) +
      Number(purchase.amount);
    await this.usersService.save(seller);
  }
}
```

SMTP : l'envoi des emails



1. Définition

- SMTP (Simple Mail Transfer Protocol) : le protocole standard d'envoi d'emails vers un serveur de messagerie.
- Notre API est un client SMTP : elle se connecte à un serveur (hôte, port, identifiants) et lui confie le message.
- MailModule s'appuie sur @nestjs-modules/mailer et Nodemailer, avec des gabarits EJS.



2. Pourquoi & Comment

POURQUOI

- Deux parcours en dépendent : vérification de l'email et mot de passe oublié.
- Un compte non vérifié ne peut pas se connecter : l'email fait partie de la sécurité.
- Envoyer de vrais emails en dev serait lent et risqué.

COMMENT

- Configuration par variables d'environnement, validées par Joi.
- En dev : conteneur Mailpit (interface web, port 8025).
- En prod : un vrai serveur SMTP, sans changer de code.



3. Exemple concret

- mail.module.ts : le transport SMTP et les gabarits.
- mail.service.ts : l'envoi du mail de vérification.
- verify-email.ejs : le gabarit du message.
- Voir la diapositive suivante.

SMTP : configuration et envoi

apps/api/src/modules/mail/mail.module.ts

```
MailerModule.forRootAsync({
  inject: [ConfigService],
  useFactory: (config: ConfigService) => {
    return {
      transport: {
        host: config.get<string>('SMTP_HOST'),
        port: config.get<number>('SMTP_PORT'),
        secure: false,
        auth: {
          user: config.get<string>('SMTP_USERNAME'),
          pass: config.get<string>('SMTP_PASSWORD'),
        },
      },
      defaults: {
        from: `"${config.get<string>('SMTP_FROM_NAME')}" <${
          config.get<string>('SMTP_FROM')}>`,
      },
      template: {
        dir: join(__dirname, 'templates'),
        adapter: new EjsAdapter({ inlineCssEnabled: true }),
      },
    };
  },
});
```

mail.service.ts

```
async sendVerifyEmailTemplate(email: string, link: string)
{
  try {
    await this.mailerService.sendMail({
      to: email,
      subject: 'Verify your account',
      template: 'verify-email',
      context: { link },
    });
  } catch (e) {
    const error = e as Error;
    this.logger.error(
      `Error sending verify email: ${error.message}`,
      error.stack,
    );
    throw new InternalServerErrorException({
      code: ErrorCodes.EMAIL_SEND_FAILED,
    });
  }
}
```

docker-compose.dev.yml

```
mailpit:
  image: axllent/mailpit:latest
  container_name: promptverse_mailpit
  ports:
    - "${MAILPIT_SMTP_PORT:-1025}:1025"
    - "${MAILPIT_UI_PORT:-8025}:8025"
```

04

Front-end

Application monopage React 19 + TypeScript



Organisation du front-end

**pages/**

Un composant par écran

**components/**

Composants réutilisables par domaine

**common/api/**

Axios, cookie envoyé automatiquement

**hooks/**

TanStack Query : données serveur

**common/store/**

Zustand : session, toasts, UI

Pourquoi TanStack Query + Zustand ?

- Cache, chargement et erreurs gérés automatiquement
- Séparation claire : données serveur ≠ état de l'interface
- Beaucoup moins de code répétitif
- Plus léger et plus simple que Redux pour ce besoin

Routes protégées et formulaires validés

components/auth/ProtectedRoute.tsx

```
export function ProtectedRoute() {
  const { status } = useAuth();
  const location = useLocation();

  if (status === "pending") {
    return <PageLoader label="Vérification de la session..." />;
  }

  if (status === "unauthenticated") {
    return (
      <Navigate
        to="/login"
        replace
        state={{ from: location.pathname + location.search }}
      />
    );
  }

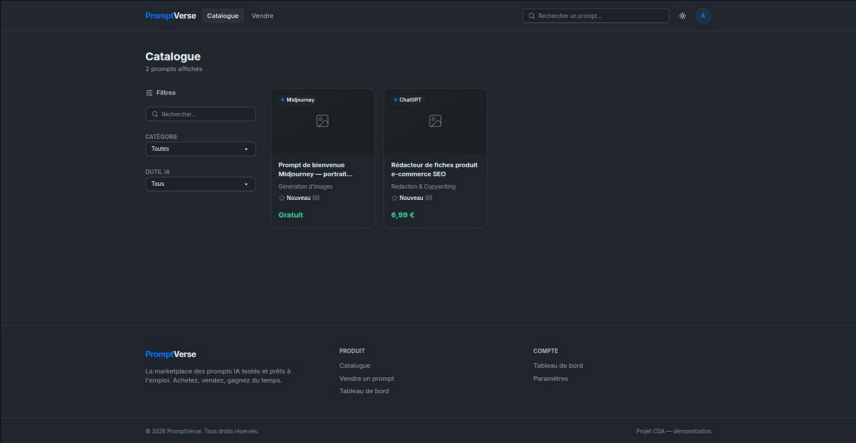
  return <Outlet />;
}
```

pages/auth/RegisterPage.tsx

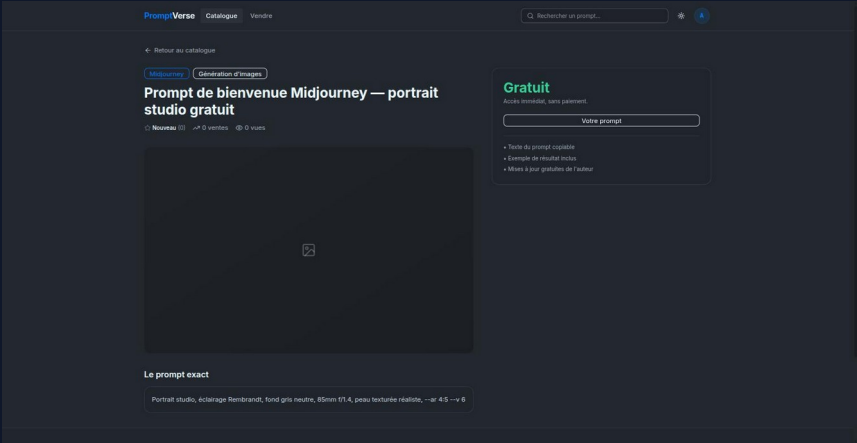
```
const schema = z
  .object({
    username: z.string().min(3, "3 caractères minimum.")
      .max(50, "50 caractères maximum."),
    email: z.string().min(1, "L'e-mail est requis.")
      .email("E-mail invalide."),
    password: z.string().min(6, "6 caractères minimum."),
    confirmPassword: z.string().min(1, "Confirmez le mot de passe."),
    terms: z.boolean()
      .refine((v) => v === true, "Vous devez accepter les conditions."),
  })
  .refine((data) => data.password === data.confirmPassword, {
    path: ["confirmPassword"],
    message: "Les mots de passe ne correspondent pas.",
  });
```

Côté client, ces contrôles servent le confort de l'utilisateur : la vraie sécurité reste dans l'API.

L'application



Catalogue filtrable



Fiche d'un prompt

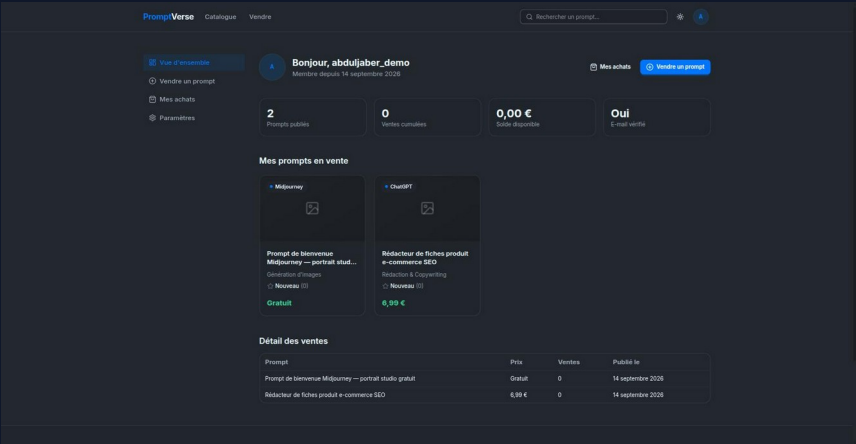
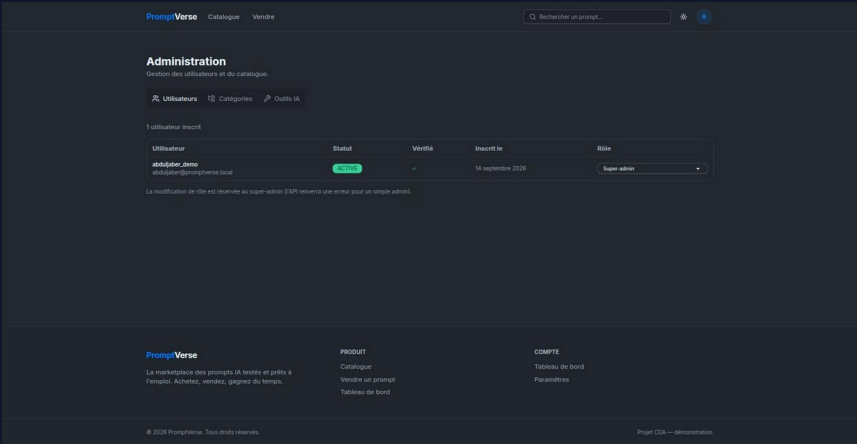


Tableau de bord vendeur



Administration des rôles

05

Sécurité

Pensée dès la conception, pas ajoutée à la fin



Mesures de sécurité



Authentification

JWT signé, cookie httpOnly + secure
+ sameSite, bcrypt



Contrôle d'accès

Guard global + rôles USER / ADMIN /
SUPER_ADMIN



Validation

DTO avec whitelist, champs inconnus
refusés



Limitation de débit

Throttler API + limites Nginx



Injection SQL

Requêtes paramétrées par TypeORM



Transport et en-têtes

HTTPS, HSTS, Helmet, CORS
restreint

RGPD et accessibilité



Données personnelles

- Collecte minimale : email, pseudo
- Mot de passe haché, jamais en clair
- Aucune donnée bancaire : tout passe par Stripe
- Acceptation des conditions à l'inscription
- Suppression du compte depuis les paramètres



Accessibilité

- Chaque champ a un libellé (label / htmlFor)
- Textes alternatifs sur les images
- Messages d'erreur en texte sous le champ
- Contrastes vérifiés (AA / AAA)
- Audit RGAA complet : à venir

06

Tests

Des critères d'acceptation aux tests automatisés



Tests unitaires et jeu d'essai

18 / 18

tests unitaires Jest réussis

10 / 11

scénarios du jeu d'essai conformes

purchases.service.spec.ts — le test d'idempotence

```
it('is idempotent: a second completed event for the same session is a no-op', async () => {
  const session = { id: 'cs_test_123' } as Stripe.Checkout.Session;
  stripeService.constructWebhookEvent.mockReturnValue({
    type: 'checkout.session.completed',
    data: { object: session },
  });
  findOnePurchase.mockResolvedValue({
    id: 'purchase-1', promptId: 'prompt-1', sellerId: 'seller-1',
    amount: '10.00', status: PurchaseStatus.COMPLETED,
  });

  await service.handleWebhookEvent(Buffer.from('{}'), 'sig');

  expect(savePurchase).not.toHaveBeenCalled();
  expect(usersService.save).not.toHaveBeenCalled();
  expect(promptsService.incrementSalesCount).not.toHaveBeenCalled();
});
```

Unitaires : services des prompts, des achats et de l'administration, avec des mocks

Jeu d'essai : vraies requêtes HTTP sur l'API (prompt gratuit, propre prompt, webhook falsifié...)

Écart analysé : clé Stripe de test invalide en local, pas un défaut du code

07

DevOps et déploiement

Du git push à la production



Conteneurisation avec Docker



Développement

docker-compose.dev.yml

- api (rechargement à chaud)
- client (Vite)
- database (MySQL 8.4)
- mailpit (emails de test)



Production

docker-compose.prod.yml

- nginx (HTTPS, reverse proxy)
- api (image Docker Hub)
- client (image Docker Hub)
- database (volume persistant)

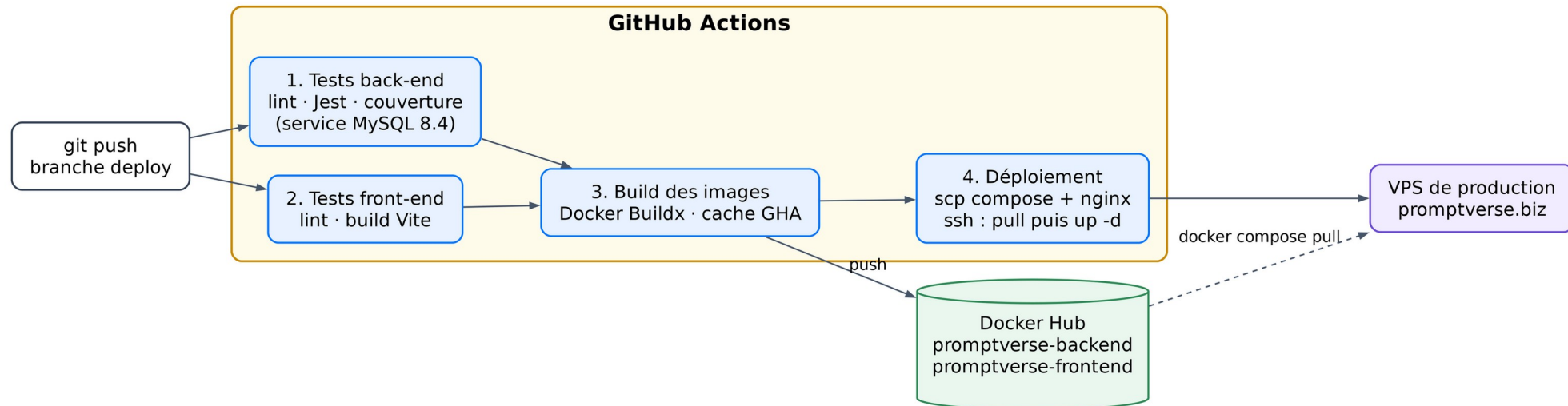
apps/api/Dockerfile (extrait)

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:22-alpine AS deps
RUN npm ci --omit=dev && npm cache clean --force

FROM node:22-alpine AS production
ENV NODE_ENV=production
COPY --from=deps /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
USER node
ENTRYPOINT ["docker-entrypoint.sh"]
CMD ["node", "dist/main"]
```

Pipeline CI/CD GitHub Actions



1 Tests back

lint · Jest · couverture

2 Tests front

lint · build Vite

3 Images

build + push Docker Hub

4 Déploiement

scp + ssh sur le VPS

Mise en production



VPS Linux

Docker Compose de production



Nginx

Seul service exposé (80 / 443)



HTTPS

Certificat Let's Encrypt via Certbot



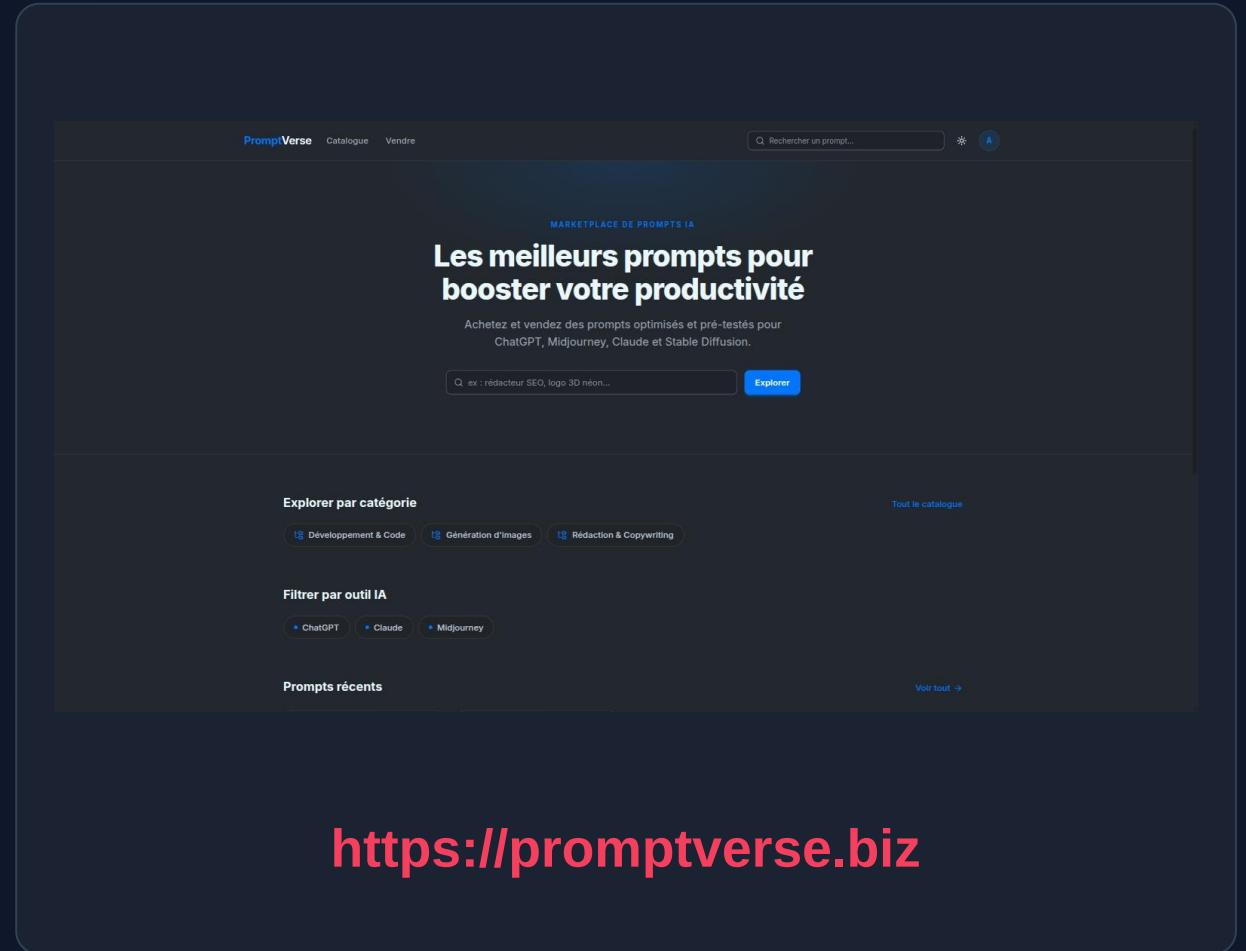
Données

Volumes MySQL et uploads persistants



Retour arrière

Images taguées par commit



<https://promptverse.biz>

Veille sur les vulnérabilités

Sources : OWASP Top 10 · CERT-FR / ANSSI · GitHub Advisory Database · npm audit

0

vulnérabilité côté client

15

alertes côté API (npm audit)



Faible trouvée dans notre code

L'envoi d'images accepte tout type « image/* », y compris le SVG, qui peut contenir du JavaScript (risque de XSS stocké). Le type MIME est en plus déclaré par le client.

Correctif prévu

- Liste blanche : JPEG, PNG, WebP uniquement
- Contrôle de la signature binaire du fichier
- Mise à jour des dépendances + npm audit dans la CI

Évolutions du projet



Avis et notes



Liste de favoris



Recherche plein texte



Reversement aux vendeurs (Stripe Connect)



Tests End-to-End et meilleure couverture



Durcissement : upload, mots de passe, dépendances

08

Déroulement

Le parcours complet, de l'inscription à l'achat



Déroulement d'un parcours complet



1. Inscription

L'utilisateur crée son compte. Le mot de passe est haché (bcrypt), un jeton de vérification valable 15 minutes est généré.



2. Email SMTP

MailService envoie le lien de vérification. En développement, Mailpit capture l'email ; en production, un vrai serveur SMTP le distribue.



3. Connexion

Une fois l'email vérifié, la connexion pose le cookie httpOnly contenant le JWT.



4. Publication

Le vendeur publie un prompt : validation par DTO, images enregistrées par Multer, mise en ligne immédiate.



5. Achat Stripe

Les règles métier sont vérifiées, un achat PENDING est créé, puis l'acheteur paie sur la page Stripe Checkout.



6. Webhook

Stripe confirme le paiement. Signature vérifiée, traitement idempotent : achat COMPLETED, vente comptée, vendeur crédité, contenu débloqué.

Chaque étape s'appuie sur ce qui précède : la spécification OpenSpec, l'email SMTP et le webhook Stripe forment un seul parcours.

DÉMONSTRATION

Place à la démo

- Inscription et vérification de l'email (Mailpit)
- Publication d'un prompt
- Achat d'un prompt avec Stripe
- Espace d'administration

promptverse.biz



Conclusion



Tout le cycle

Des besoins à la production, en passant par UML, Merise, le code, les tests et le déploiement



La sécurité dès la conception

Cookie httpOnly, routes privées par défaut, contenu payant masqué, webhook idempotent



Le travail en équipe

Backlog partagé, un dépôt commun, des décisions prises à trois



L'esprit DevOps

Un projet est terminé quand il tourne en production, de façon reproductible

Merci pour votre attention

Place à vos questions



Amine LAMALLEM · PromptVerse · promptverse.biz